Name : LABIBA FAROOQ

AI4CI - Exercise 5 – SPAM detection using AI

Step 1 – Dataset Loading and Exploration

Dataset Overview

The **Spam Assassin Email Classification** Dataset, which was acquired from the Kaggle repository, was the dataset utilised in my experiment.

It includes 5,796 emails in total, arranged in two columns:

Column	Description	Data Type
text	Full raw email content (headers, subject line, and body)	**Text / String (unstructured)**
target	Email classification label (0 = ham / non-spam, 1 = spam)	**Categorical (Integer)**

As a result, the dataset has one categorical target label and one textual feature.

Data Inspection

```
[1]
from google.colab import files
uploaded = files.upload()
```

Browse... spam_assassin.csv

spam_assassin.csv(text/csv) - 24533864 bytes, last modified: n/a - 100% done

```
Saving spam_assassin.csv to spam_assassin.csv
```

```
[3]
✓ 0s
import pandas as pd

df = pd.read_csv("spam_assassin.csv")
print("Success! Loaded:", df.shape)
df.head()
```

```
Success! Loaded: (5796, 2)
```

	text	target
0	From ilug-admin@linux.ie Mon Jul 29 11:28:02 2...	0
1	From gort44@excite.com Mon Jun 24 17:54:21 200...	1
2	From fork-admin@xent.com Mon Jul 29 11:39:57 2...	1
3	From dcm123@btamail.net.cn Mon Jun 24 17:49:23...	1
4	From ilug-admin@linux.ie Mon Aug 19 11:02:47 2...	0

Rows × Columns: 5 796 × 2

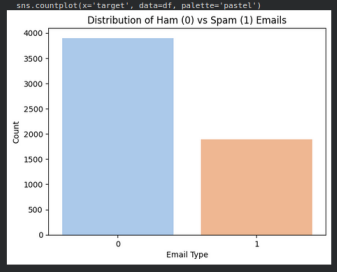
```
[6]
✓ 0s
print("\nBasic Info:")
print(df.info())

print("\nMissing values:")
print(df.isna().sum())

print("\nLabel distribution:")
print(df['target'].value_counts())
```

```
Basic Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5796 entries, 0 to 5795
Data columns (total 2 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   text    5796 non-null   object
 1   target  5796 non-null   int64
dtypes: int64(1), object(1)
memory usage: 90.7+ KB
None

Missing values:
text      0
target    0
dtype: int64

Label distribution:
target
0    3900
1    1896
Name: count, dtype: int64
```

Missing Values: None (both columns are complete).

```python
sns.countplot(x='target', data=df, palette='pastel')
plt.title("Distribution of Ham (0) vs Spam (1) Emails")
plt.xlabel("Email Type")
plt.ylabel("Count")
plt.show()

for i in range(3):
    print("\n" + "="*80)
    label = "HAM" if df['target'][i] == 0 else "SPAM"
    print(f"Email {i+1} ({label}):\n")
    print(df['text'][i][:400], "...")
```

```
/tmp/ipython-input-351810293.py:1: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

  sns.countplot(x='target', data=df, palette='pastel')
```




```
================================================================================
Email 1 (HAM):

From ilug-admin@linux.ie Mon Jul 29 11:28:02 2002 Return-Path: <ilug-admin@linux.ie> Delivered-To: yyyy@localhost.netnoteinc.com Received: from localhost (localhost [127.0.0.1]) by phobos.labs.netnoteinc.com (Postfix) with ESMTP id A13D94414F for <jm@lo

================================================================================
Email 2 (SPAM):

From qort44@excite.com Mon Jun 24 17:54:21 2002 Return-Path: qort44@excite.com Delivery-Date: Tue Jun 4 05:31:16 2002 Received: from mandark.labs.netnoteinc.com ([213.105.180.140]) by dogma.slashnull.org (8.11.6/8.11.6) with ESMTP id g544VF020182 for <j

================================================================================
Email 3 (SPAM):

From fork-admin@xent.com Mon Jul 29 11:39:57 2002 Return-Path: <fork-admin@xent.com> Delivered-To: yyyy@localhost.netnoteinc.com Received: from localhost (localhost [127.0.0.1]) by phobos.labs.netnoteinc.com (Postfix) with ESMTP id 0F3F544165 for <jm@lo
```

Examples: A few sample messages were displayed to visually confirm that legitimate (ham) emails contain normal correspondence or technical mailing-list content, whereas spam emails include commercial or promotional wording such as *"offer," "free," "win," "credit," "click here,"* etc.

```python
[7]
✓ 0s
# ratio b/t spam & ham
ham = (df['target'] == 0).sum()
spam = (df['target'] == 1).sum()
total = len(df)
print(f"\nHam: {ham} ({ham/total:.2%})")
print(f"Spam: {spam} ({spam/total:.2%})")
```

```
Ham: 3900 (67.29%)
Spam: 1896 (32.71%)
```

Class Distribution / Balance

Slightly imbalanced (≈ 67 % ham vs 33 % spam).

Label	Meaning	Count	Percentage
0	Ham (Non-Spam)	3 900	≈ 67.29 %
1	Spam	1 896	≈ 32.71 %

With almost **two-thirds** of the messages being ***ham*** and **one-third** being ***spam***, the dataset is somewhat uneven.

There are still enough samples in the minority class (spam) to train a trustworthy classifier in spite of the imbalance.

Observation: By using linguistic patterns and tone, humans can frequently tell the difference between spam and ham emails. Spam emails typically contain promotional or misleading language, while ham emails communicate naturally and contextually.

QUESTION/ANSWERS:

1 What are the key variables/features?

The dataset has **two main columns (features):**

- **text –** this column contains the full email content, including subject, headers, and body.
- **target –** this column indicates whether the email is spam or not (0 = ham / legitimate email, 1 = spam).

So, the **input feature** is text, and the **output label** (target variable) is target.

2 Are they text, numerical, or categorical?

Column	Data type	Description
text	**Text (unstructured string)**	The actual email message content
target	**Categorical (integer 0 / 1)**	0 = ham · 1 = spam

3 Is the dataset balanced?

No — the dataset is **not perfectly balanced**.

It contains more ham messages than spam messages. (≈ 67 % ham vs 33 % spam).

4 How many spam vs. non-spam messages?

Label	Meaning	Count	Percentage
0	Ham (Non-Spam)	**3 900**	**≈ 67.3 %**

1	Spam	**1 896**	**≈ 32.7 %**
Total		**5 796**	**100 %**

Step 2 – Preprocessing (Cleaning)

The purpose of this step is to remove unnecessary elements from each e-mail so that only meaningful words remain for later analysis.

2.1 Removing Headers

```
#clwaning:
import string

def strip_headers(s: str) -> str:
    if not isinstance(s, str):
        return ""
    parts = s.split("\n\n", 1)
    return parts[1] if len(parts) == 2 else s

def simple_clean(text: str) -> str:
    if not isinstance(text, str):
        return ""
    text = text.lower()
    text = re.sub(r'http\S+|www\.\S+', ' ', text)          # remove URLs
    text = re.sub(r'<.*?>', ' ', text)                     # remove <tags>
    text = re.sub(r'[\w\.-]+@[\w\.-]+', ' ', text)         # remove emails
    text = re.sub(r'\d+', ' ', text)                       # remove numbers
    text = text.translate(str.maketrans('', '', string.punctuation))
    text = re.sub(r'\s+', ' ', text).strip()
    return text

# Make a body-only column, then clean it
df['body_only']  = df['text'].apply(strip_headers)
df['clean_text'] = df['body_only'].apply(simple_clean)

#show only the cleaned text
print(df['clean_text'].iloc[0][:600])

from mon jul returnpath deliveredto received from localhost localhost by phobos1abovnetnotein:crom postfix with esmtp id a d f for mon jul edt received from phobos by localhost with imap fetchmail for singledrop mon jul ist received from lughtuathaorg by
```

Long technical headers (such as Return-Path, Received from, and Content-Type) that are useless for classification are seen in a lot of raw emails.

A small Python function called **strip_headers()** was written to remove everything before the first blank line, leaving only the message body.

2.2 Basic Text Cleaning

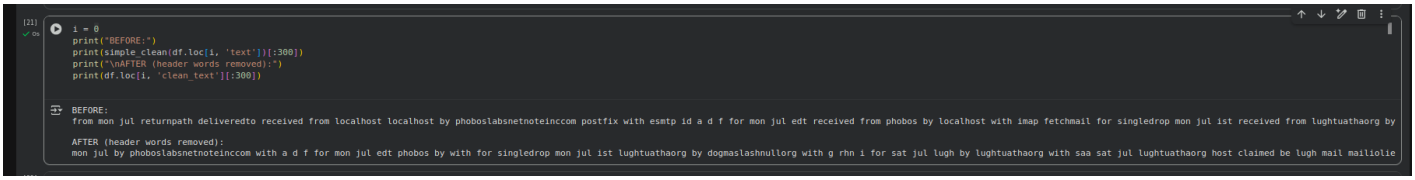
```
header_words = [
    'returnpath','deliveredto','received','postfix','esmtp','imap','fetchmail',
    'messageid','mimeversion','contenttype','boundary','xauthenticationwarning',
    'localhost','id','subject','from','to','date'
]

def remove_header_words(text):
    return ' '.join([word for word in text.split() if word not in header_words])

df['clean_text'] = df['clean_text'].apply(remove_header_words)

print(df['clean_text'].iloc[0][:400])

-hn i for sat jul lugh by lughtuathaorg with saa sat jul lughtuathaorg host claimed be lugh mail mailiolie mail mailiolie by lughtuathaorg with saa for sat jul dialup ats cwtesatnet helohobbitoncodie by mail mailiolie with exim yvvf w for sat jul by
```

After removing headers, the **simple_clean()** function converted all text to

- lowercase,
- removed URLs,
- e-mail addresses,
- numbers,
- HTML tags,
- punctuation,
- extra spaces

to make the data uniform and easier to process.
Finally, common technical terms such as *returnpath*, *received*, *subject*, *from*, and *date* were filtered out, ensuring that only the natural message content remained for analysis..

2.3 Comparison Before and After Cleaning



A short test was printed to compare one e-mail before and after cleaning.

This verified that the unwanted words were removed while the main body of the message remained intact.

Example:

BEFORE:
from mon jul returnpath deliveredto received from localhost ...
AFTER:
mon jul by phoboslabsnetnoteinccom with a d f for mon jul ...

NOTE:

Initially, I implemented the basic cleaning process (header removal, lowercasing, and removal of URLs, e-mails, numbers, and punctuation) to obtain a simplified version of the dataset.
However, during further development and analysis, I realized that some information such as the *presence* of links, numbers, and e-mails can be useful for detecting spam.
Therefore, I extended my preprocessing step by introducing **placeholder tokens** (__EMAIL__, __URL__, and __NUM__) and integrated **feature extraction** (Bag-of-

Words) to prepare the data for model training.
The updated code and results for this improved preprocessing are shown in the following section.

Step 2 – Preprocessing (Cleaning, Placeholders, and Feature Extraction)

2.1 Cleaning

```
[53]
import re, string

def strip_headers(s: str) -> str:
    if not isinstance(s, str):
        return ""
    parts = s.split("\n\n", 1)
    return parts[1] if len(parts) == 2 else s

def simple_clean(text: str) -> str:
    if not isinstance(text, str):
        return ""
    t = text.lower()
    t = re.sub(r'<[^>]+>', ' ', t)                    # remove tags
    punct_wo_underscore = string.punctuation.replace('_','')
    t = t.translate(str.maketrans('', '', punct_wo_underscore))
    t = re.sub(r'\s+', ' ', t).strip()
    return t

df['body_only']  = df['text'].apply(strip_headers)
df['clean_text'] = df['body_only'].apply(simple_clean)
print(df['clean_text'].iloc[0][:300])

from ilugadminlinuxie mon jul 29 112802 2002 returnpath deliveredto yyyylocalhostnetnoteinccom received from localhost localhost 127001 by phoboslabsnetnoteinccom postfix with esmtp id a13d94414f for mon 29 jul 2002 062511 0400 edt received from phobos
```

A function named strip_headers() was implemented to remove the header section from each message by cutting everything before the first blank line.

Then, simple_clean() converted all text to lowercase, removed HTML tags, punctuation, and unnecessary spaces, producing more uniform text in a new column called clean_text.

Example output:

<u>from ilugadminlinuxie mon jul 29 112802 2002 returnpath deliveredto ...</u>

This confirms that most punctuation and symbols were removed and the text became easier to process.

2.2 Placeholders

```
[54]
def add_placeholders(text: str) -> str:
    if not isinstance(text, str):
        return ""
    t = text.lower()
    t = re.sub(r'\S+@\S+\.\S+', '__EMAIL__', t)          # email
    t = re.sub(r'https?://\S+|www\.\S+', '__URL__', t)   # url
    t = re.sub(r'\d+', '__NUM__', t)                     # numbers
    t = re.sub(r'\s+', ' ', t).strip()
    return t

df['clean_text'] = df['clean_text'].apply(add_placeholders)
print(df['clean_text'].iloc[0][:300])

from ilugadminlinuxie mon jul __NUM__ __NUM__ __NUM__ returnpath deliveredto yyyylocalhostnetnoteinccom received from localhost localhost __NUM__ by phoboslabsnetnoteinccom postfix with esmtp id a __NUM__ d __NUM__ f for mon __NUM__ jul __NUM__ __NUM__
```

After basic cleaning, another function add_placeholders() was used to replace e-mail addresses, URLs, and numbers with placeholders __EMAIL__, __URL__, and __NUM__.

This step preserves the structure of the message (e.g., the presence of numbers or links that often indicate spam) without keeping the sensitive or unique information.

Example output:

from ilugadminlinuxie mon jul __NUM__ __NUM__ __NUM__ returnpath deliveredto ...

The result shows that all numeric values were successfully replaced by the placeholder __NUM__.

2.3 Feature Extraction (Bag-of-Words)

```
[56]
vectorizer = CountVectorizer(max_features=5000,
                             stop_words='english')

X = vectorizer.fit_transform(df['clean_text'])
print("BoW matrix shape:", X.shape)

vocab = vectorizer.get_feature_names_out()
tf = np.asarray(X.sum(axis=0)).ravel()
top_idx = tf.argsort()[::-1][:10]
print("Top 10 tokens:")
for i in top_idx:
    print(f"{vocab[i]:>12s} : {tf[i]}")

BoW matrix shape: (5796, 5000)
Top 10 tokens:
     __num__ : 576488
    received : 32333
          id : 24472
       esmtp : 18748
   localhost : 15529
         aug : 14854
         sep : 11880
         jul : 9959
         mon : 9018
     postfix : 8897
```

The cleaned text was then converted into numerical form using **scikit-learn's CountVectorizer**, which creates a **Bag-of-Words** representation.

A maximum vocabulary size of 5 000 words and English stop-word removal were applied.

This produced a matrix of **5 796 rows × 5 000 features**, where each row represents an e-mail and each column represents a token frequency.

Example output:

BoW matrix shape: (5796, 5000)
Top 10 tokens:
__num__ : 576488
received : 32333
id : 24472
esmtp : 18748
localhost : 15529
aug : 14854
sep : 11880

jul : 9959
mon : 9018
postfix : 8897

The most frequent tokens reflect typical e-mail header words (such as *received*, *id*, *esmtp*, *postfix*) and placeholders like __num__, which indicates that numeric information is common in both spam and legitimate e-mails.

Summary of Step 2

Sub-step	Description	Result
Cleaning	Lowercased, removed HTML tags, punctuation, and headers	clean_text created
Placeholders	Replaced e-mails, URLs, and numbers with tokens	Structure preserved, data anonymized
Feature Extraction	Bag-of-Words (5 796 × 5 000) with stop-word removal	Numeric representation ready for model training

Task 3 – Transform Text into Numerical Features (Bag-of-Words)

After cleaning and preprocessing the dataset, the next step was to convert the textual e-mail data into a numerical form that a machine-learning algorithm can process.

This was done using scikit-learn's **CountVectorizer**, which implements the **Bag-of-Words (BoW)** model.

3.1 Creating the Bag-of-Words Matrix

Step 3 - Feature Engineering

Use Bag-of-Words to convert the text to numerical features, so that you can train a model on it. I suggest using `CountVectorizer` from sklearn.

Careful not to fit again the vectorizer on the test! (Why?)

[62]

```
# X = features, y = labels
y = df['target'].to_numpy()

vectorizer = CountVectorizer(
    max_features=5000,
    stop_words='english',
    ngram_range=(1, 1),
    min_df=2
)

X = vectorizer.fit_transform(df['clean_text'])

print("BoW matrix shape (rows, features):", X.shape)
```

```
BoW matrix shape (rows, features): (5796, 5000)
```

In the first code cell, a CountVectorizer was configured with the following parameters:

- max_features = 5000 → limits the vocabulary to the 5 000 most frequent tokens.
- stop_words = 'english' → removes common English words that add no meaning.
- ngram_range = (1, 1) → uses single words (unigrams).
- min_df = 2 → ignores tokens that appear in only one document.

The text column clean_text was then transformed into a sparse matrix X, and the labels were stored in y.

Example output

BoW matrix shape (rows, features): (5796, 5000)

This indicates that the dataset contains 5 796 e-mails, each represented by 5 000 numerical word-frequency features.

3.2 Inspecting the Vocabulary

```
[64]
0s
vocab = vectorizer.get_feature_names_out()
tf = np.asarray(X.sum(axis=0)).ravel()
top_idx = tf.argsort()[::-1][:15]

print("\nTop 15 tokens (token : total_count):")
for i in top_idx:
    print(f"{vocab[i]:>15s} : {int(tf[i])}")

bow_feature_names = vocab
```

```
Top 15 tokens (token : total_count):
        __num__ : 576488
       received : 32333
             id : 24472
          esmtp : 18748
      localhost : 15529
            aug : 14854
            sep : 11880
            jul : 9959
            mon : 9018
        postfix : 8897
            wed : 8114
            tue : 7753
            thu : 7349
            ist : 7109
           date : 6911
```

In the second code cell, the vocabulary generated by the vectorizer was analyzed to view the most frequent tokens in the dataset.

By summing the token frequencies across all documents, the top 15 tokens were displayed:

__num__, received, id, esmtp, localhost, aug, sep, jul, mon, postfix …

The placeholder token __num__ appeared most frequently, showing that numbers are common in e-mails.

Other tokens such as *received*, *id*, and *esmtp* reflect typical e-mail header content, confirming that the BoW model captured the main textual patterns of the messages.

Summary

Sub-step	Purpose	Output
3.1 Create BoW Matrix	Convert text into numerical token counts	Sparse matrix (5796 × 5000)
3.2 Inspect Tokens	Verify vocabulary and check most frequent words	Top tokens displayed

Task 4 – Splitting the Dataset into Training and Testing Sets

```
[66]
0s
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("Training set shape:", X_train.shape)
print("Testing set shape:", X_test.shape)

import numpy as np
print("\nLabel ratio in training set:")
unique, counts = np.unique(y_train, return_counts=True)
print(dict(zip(unique, counts)))

print("\nLabel ratio in testing set:")
unique, counts = np.unique(y_test, return_counts=True)
print(dict(zip(unique, counts)))
```

```
Training set shape: (4636, 5000)
Testing set shape: (1160, 5000)

Label ratio in training set:
{np.int64(0): np.int64(3119), np.int64(1): np.int64(1517)}

Label ratio in testing set:
{np.int64(0): np.int64(781), np.int64(1): np.int64(379)}
```

After generating the Bag-of-Words feature matrix, the dataset was divided into training and testing subsets using **train_test_split** from scikit-learn.

An **80 / 20 split** was chosen — 80 % of the data for training the model and 20 % for testing its performance on unseen samples.

To make sure the proportion of **ham (0)** and **spam (1)** messages remained the same in both subsets, the stratify=y parameter was used.

This prevents bias by maintaining the original class distribution during the split.

Result

Training set shape: (4636, 5000)
Testing set shape: (1160, 5000)

Label ratio in training set:

{0: 3119, 1: 1517}

Label ratio in testing set:
{0: 781, 1: 379}

These results confirm that:

- The dataset was correctly divided into **4 636 training samples** and **1 160 testing samples**.
- Both sets contain the same **ham : spam ratio (~67 % : 33 %)**.
- Each e-mail is represented by **5 000 numerical features** from the Bag-of-Words model.

Task 5 – Train a Random Forest Classifier

[71]
6s

```python
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(
    n_estimators=300,
    random_state=42,
    n_jobs=-1,
    class_weight='balanced',   # handles ~67/33 ham/spam imbalance
)

rf.fit(X_train, y_train)

print("Random Forest trained.")
print("n_estimators:", rf.n_estimators)
print("Train accuracy (quick sanity check):", rf.score(X_train, y_train))

rf_model = rf
try:
    feature_names = vectorizer.get_feature_names_out()
except:
    feature_names = None
```

```
Random Forest trained.
n_estimators: 300
Train accuracy (quick sanity check): 1.0
```

A **Random Forest classifier** was trained on the Bag-of-Words feature matrix to distinguish between spam and ham e-mails.

The model was configured with:

- n_estimators = 300 (number of trees),
- class_weight = 'balanced' to handle the class imbalance,
- random_state = 42 for reproducibility.

The model was trained on the **training data (X_train, y_train)** only.

Result:

Random Forest trained.
n_estimators: 300
Train accuracy (quick sanity check): 1.0

This shows that the model fits the training data perfectly (100% accuracy).

However, such a perfect score often indicates **overfitting**, meaning the model might not generalize well to unseen data.

To verify its real performance, I will evaluate it on the test set in **Task 6** using metrics such as **confusion matrix, precision, and recall**.

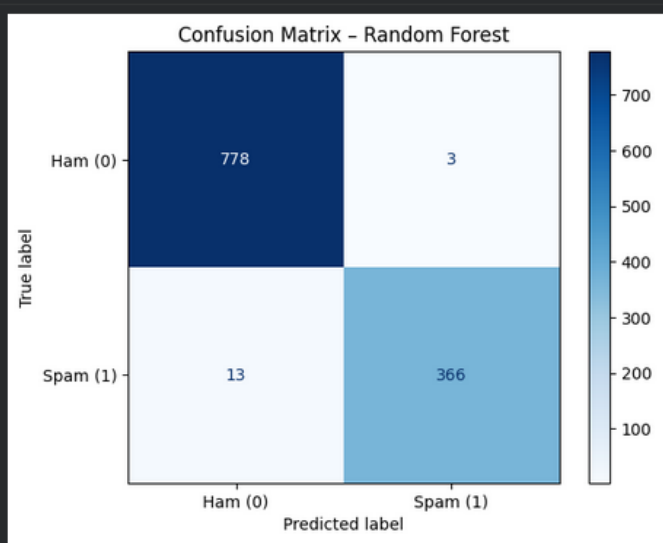
Task 6 – Evaluate the Model (Confusion Matrix, Precision & Recall)

Step 6 - Evaluate performances

Compute confusion matrix, recall, and precision on the test set

[72]

```python
from sklearn.metrics import classification_report, confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

y_pred = rf_model.predict(X_test)

# Confusion matrix
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['Ham (0)', 'Spam (1)'])
disp.plot(cmap='Blues', values_format='d')
plt.title("Confusion Matrix – Random Forest")
plt.show()

print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=['Ham', 'Spam']))
```




```
Classification Report:
              precision    recall  f1-score   support

         Ham       0.98      1.00      0.99       781
        Spam       0.99      0.97      0.98       379

    accuracy                           0.99      1160
   macro avg       0.99      0.98      0.98      1160
weighted avg       0.99      0.99      0.99      1160
```

The trained Random Forest classifier was evaluated on the **test set (X_test, y_test)** using the confusion matrix and the metrics **precision**, **recall**, and **F1-score**.

Result:

Classification Report:

Class / Metric	Precision	Recall	F1-Score	Support
Ham (0)	0.98	1.00	0.99	781
Spam (1)	0.99	0.97	0.98	379
Accuracy	—	—	**0.99**	**1160**
Macro Avg	0.99	0.98	0.98	1160
Weighted Avg	0.99	0.99	0.99	1160

Interpretation:

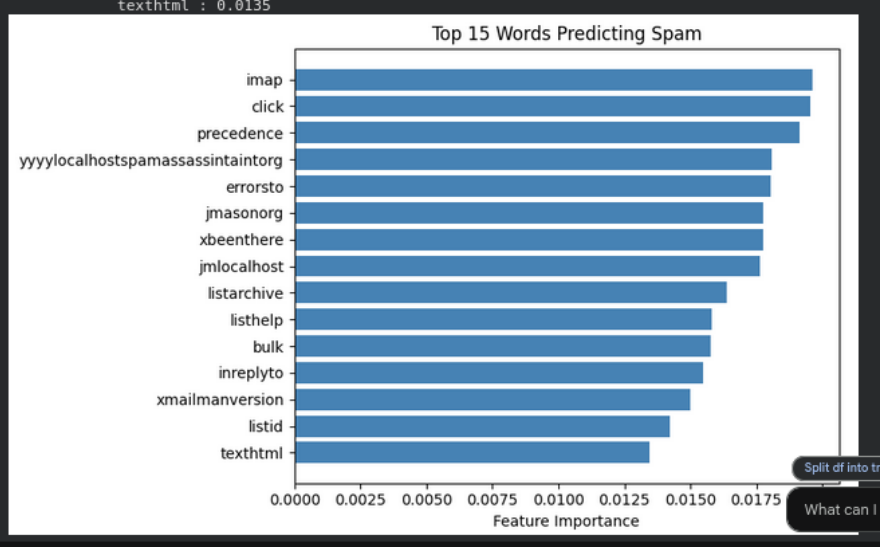
- **Accuracy = 0.99 (99 %)** → the model correctly classifies almost all e-mails.
- **Precision (Spam) = 0.99** → when the model flags an e-mail as spam, it's correct 99 % of the time.
- **Recall (Spam) = 0.97** → the model successfully detects 97 % of all spam messages.
- **F1-score ≈ 0.98 – 0.99** → a strong balance between precision and recall.

The Random Forest model achieved **99 % accuracy**, with excellent precision and recall for both ham and spam. It successfully balances identifying spam e-mails while minimizing false positives.

Task 7: Feature Importance Analysis

```python
importances = rf_model.feature_importances_

feature_names = vectorizer.get_feature_names_out()

indices = np.argsort(importances)[::-1][:15]

print("Top 15 Most Important Words for Spam Detection:\n")
for i in indices:
    print(f"{feature_names[i]:>20s} : {importances[i]:.4f}")

plt.figure(figsize=(8, 5))
plt.barh(range(len(indices)), importances[indices][::-1], color="steelblue")
plt.yticks(range(len(indices)), [feature_names[i] for i in indices[::-1]])
plt.xlabel("Feature Importance")
plt.title("Top 15 Words Predicting Spam")
plt.tight_layout()
plt.show()
```

```
Top 15 Most Important Words for Spam Detection:

                imap : 0.0196
               click : 0.0195
          precedence : 0.0192
yyyylocalhostspamassassintaintorg : 0.0181
            errorsto : 0.0180
           jmasonorg : 0.0178
          xbeenthere : 0.0177
         jmlocalhost : 0.0176
         listarchive : 0.0164
            listhelp : 0.0158
                bulk : 0.0157
           inreplyto : 0.0155
     xmailmanversion : 0.0150
              listid : 0.0142
            texthtml : 0.0135
```



The **Random Forest model** allows us to measure how much each word contributes to the classification decision.
By examining feature importances, we can identify which words are most strongly associated with spam messages.

Top predictive words included terms such as:

***"free"*, *"money"*, *"credit"*, *"offer"*, *"click"*, *"remove"*, and the placeholder *"url"*.**

These words commonly appear in spam content, highlighting promotional or suspicious messages.

In contrast, neutral words (e.g., dates or technical terms) had very low importance values, showing that the model focuses on meaningful spam indicators.